Review of 8th May, 2026

1. Invite + Registration State Errors

Critical issue:
The system says “Failed to accept an invite” even though the invite actually succeeded.

Observed behavior:
▪ User clicks invite link -> from email
▪ System throws error
▪ Backend actually accepts invite
▪ User is added to pod/event anyway

This creates false-negative state messaging.

Problem:
Frontend and backend are not synchronized.

Fix:
▪ Invite acceptance must return one authoritative state
▪ If accepted successfully:
→ auto redirect into pod/event
→ show success state only

▪ Never show failure if backend succeeded
▪ Invite links must be idempotent:
→ already accepted = “You are already registered”
→ not an error

2. Matching Engine Uses Wrong Participant Source

Critical logic bug.

Observed:
Matching engine included users not present in the main room.

Example:
Wazim was included in breakout allocation despite not being actively present.

Root cause:
Matching engine currently pulls:
“all registered participants”

Instead of:
“currently connected active participants in main room”

Fix:
Matching should ONLY include:
▪ connected users
▪ currently present in lobby/main room
▪ not disconnected
▪ not ghost sessions
▪ not stale websocket connections

Required logic:
Event Participant
≠
Connected Active Participant

Those are different datasets.

4. Frontend Not Reflecting Backend State

Repeated multiple times.

Examples:
▪ co-host assignment works in backend but not UI
▪ refresh required to see actual state
▪ UI says no host despite host existing
▪ controls freeze
▪ clicks stop responding
▪ room allocations visually incorrect

Direct quote insight:
“The backend is working but the frontend is not.”

This is a reactive state synchronization issue.

Likely causes:
▪ stale frontend cache
▪ websocket state not rebroadcasting
▪ missing optimistic updates
▪ missing event listeners

Fix:
All host actions must:
▪ update backend
▪ broadcast event
▪ instantly rerender frontend state

Without refresh.

5. Control Center Architecture Is Confused

Big UX architecture issue.

Current system has:
▪ controls partly inside room UI
▪ partly inside Control Center
▪ overlapping functionality

Result:
Hosts do not know where event management actually happens.

Stefan's point was correct:
The host should manage the event from ONE operational control surface.

Fix:
Control Center becomes the single source of truth for:
▪ room management
▪ participant movement
▪ co-host management
▪ rematching
▪ round control
▪ timers -> in a sense host has control to add time to the session
▪ overrides
▪ room visibility

Main room UI should only contain:
▪ essential hot actions
▪ speaking/video interface

6. Drag and Drop Room Management Is Broken in terms of matching people with swap or drag

Observed:
▪ drag not working
▪ swap partially working
▪ modal gets stuck
▪ cannot close window
▪ cannot scroll
▪ controls freeze

This is currently unstable.

Fix:
Either:
A) fully implement drag/drop properly

OR

B) temporarily remove it completely and use:
▪ dropdown reassignment
▪ swap buttons
▪ assign-to-room actions

Half-working drag systems destroy confidence very fast.

7. Matching Logic Produces Invalid Room Structures

Observed:
▪ 7 users generated wrong room counts
▪ co-host logic inconsistent
▪ host exclusion inconsistent
▪ participants duplicated
▪ users matched with themselves effectively

Fix:
Matching engine needs hard validation rules before execution.

Required validations:
▪ one user = one room only
▪ room size validation
▪ no empty rooms
▪ no duplicate allocations
▪ host/co-host inclusion rules explicit
▪ disconnected users excluded
▪ stale users excluded

If validation fails:
DO NOT START BREAKOUTS

8. Chat System Bugs

Observed:
▪ break out room chat not updating properly
▪ messages visually appear but functionality broken

Fix:
▪ reliable realtime updates
▪ message delivery acknowledgment
▪ proper room-scoped chat state

9. Co-host System Incomplete

Observed:
▪ assigning co-host inconsistent
▪ removing co-host inconsistent

▪ frontend doesn’t update immediately
▪ unclear if multiple co-hosts supported

Current behavior feels accidental rather than intentionally designed.

Need explicit rules:
▪ how many co-hosts allowed?
▪ can co-hosts join matching?
▪ can co-hosts control rooms?
▪ can co-hosts override matches?
▪ hierarchy between host/co-host?

10. “Test Mode” Has No Defined Product Purpose

This is important.

Nobody could explain:
▪ what it does
▪ why it exists
▪ what changes when enabled

That means it should not exist yet.

Rule:
If feature purpose cannot be explained in one sentence:
remove it until defined properly.

11. Host View UX Problems

Observed:
▪ compact mode inconsistent
▪ host visibility logic unclear
▪ host video sizing inconsistent
▪ host operational focus unclear

Correct insight from meeting:
Hosts do not primarily need participant immersion.

Hosts need:
▪ operational clarity
▪ room visibility
▪ control access
▪ event status awareness

Host mode should prioritize:
▪ controls
▪ room states
▪ participant states

- timers
- alerts

Basically,

System automatically puts host and co hosts on pin on compact or in one of the spaces in the main room.